

On the Pitfalls of Modelling Individual Knowledge^{*}

Wojciech Ciszewski¹, Stefan Dziembowski^{1,2}, Tomasz Lizurej³, and Marcin Mielniczuk¹

¹ University of Warsaw

² IDEAS Institute

³ NASK Institute

Abstract. The concept of *knowledge* has been central in cryptography, especially within cryptographic proof systems. Traditionally, research in this area considers an abstract *prover* defending a claim that it knows a message M . Recently, a stronger concept—termed “individual” (Dziembowski et al., CRYPTO’23) or “complete” (Kelkar et al., CCS’24) knowledge—has emerged. This notion ensures the prover *physically stores* M on a machine that it controls. As we argue in the paper, this concept also appears in earlier work on “non-outsourcable puzzles” (Miller et al., CCS’15), which implicitly assumes that performing quickly complex computation on a string M implies storing it on a single machine.

In this line of work, the authors typically rely on the algorithms whose computation requires a massive number of queries to a hash function H . This paper highlights a subtle issue in the modeling used in some of these papers, more concretely, the assumption that H can be modeled as an *atomic* random oracle on long messages. Unfortunately, this does not correspond well to how the hash functions are constructed in practice. For example, the real-world hash functions (e.g., Merkle-Damgård or sponge-based) allow partial evaluation on long inputs, violating this assumption. Another example is the hashing used in Bitcoin mining, which permits similar precomputation. This undermines some protocols relying on individual knowledge. We demonstrate practical attacks against Miller et al.’s and Kelkar et al.’s schemes based on this observation, and discuss secure alternatives. Our alternative constructions, which are modifications of the original ones, avoid reliance on the random oracle behavior of hash functions on long messages. In the full version of this paper, we will provide their formal security analysis in the individual cryptography model of Dziembowski et al. (CRYPTO’23).

1 Introduction

The challenge of *proving knowledge* has been a topic of significant interest in cryptography, starting from the seminal work of Goldwasser, Micali, and Rackoff [20]. Generally speaking, in this line of work, the goal is to design protocols that allow a *prover* to convince a *verifier* that it *knows* a specific message M . In the traditional approach, the prover is considered an abstract entity, and the standard definitions do not guarantee that M is actually present in the memory of a single machine in a way that makes it accessible to its owner. For example, it is perfectly fine for the prover to be emulated by a group of machines, where different machines hold parts of the message M , or even a share of M using a secret sharing scheme [36]. It is also ok for the prover to have M hidden within a trusted execution environment (TEE, see, e.g., [40]). Perhaps surprisingly, some recent works considered an intriguing stronger notion, which guarantees that the prover has physical access to data M , even in the presence of malicious provers that can split their computation across multiple machines using secure multiparty computation (MPC) protocols (see, e.g., [11]), or use the TEEs. This concept has been named *proofs of individual* [16] (or *complete* [24]) *knowledge*. Applications of this stronger notion include deniable messaging, electronic voting, and prevention of account sharing.

On a high level, the solution of [16, 24] is based on requiring the prover to compute a function f (on input M) that is designed in such a way that the prover must perform the computation of f *individually*, i.e., they must have free access to M stored in its memory. More concretely, the function f is built using a hash function H , modeled as a random oracle [4], and the protocol requires the prover to compute a massive

^{*} This work has been partially funded by the European Research Council (ERC) under the European Union’s Horizon 2020 innovation program (grant PROCONTRA-885666).

number of hashes H within some time limit. This limit is set in such a way that it is infeasible to compute all the hashes without having at least some of their inputs in the memory of a single machine. This should hold both against a set of adversaries that “emulate” a single-machine prover by running an MPC protocol, or against those that do it using the TEEs. In the first case, the assumption is justified by the fact that computing a hash in MPCs is significantly slower than computing it on a single machine. In the second case, the assumption corresponds to the fact that no TEE-equipped processor can compete in the efficiency of massive parallel hash computations with hardware explicitly designed for this purpose (e.g., the ASICs used in Bitcoin mining).

As we explain below, similar flavor of knowledge assumptions (although without considering attacks that use MPCs or TEEs) have been implicitly used in the past in the line of work on “nonoutsourcable puzzles” initiated by Miller et al. [30], which are *proof-of-work* (PoW) protocols [14] designed to discourage the creation of mining pools in cryptocurrencies. The main idea of [30] is to design a PoW mining protocol that discourages creating *mining pools*. Recall that in a Bitcoin mining pool (see, e.g., [33]), the *pool operator* outsources the mining work to the *pool members* (the “workers”), who are paid for their work. The nonoutsourcable puzzles protocol of [30] is designed in such a way that every pool member can “steal” the correct puzzle solution that they found. This mechanism discourages mining pool creation, as no pool operator can be guaranteed to receive the solutions from its members.

The main idea of the protocol of [30] is as follows. Their puzzle is designed in such a way that, to find a solution, the prover must know a random nonce r (cf. Fig. 1 in Sec. 3). On the other hand, knowing r suffices to steal the solution from the pool operator. Note that breaking the non-outsourcability of such a scheme would correspond to computing the solution without having the entire r in the memory of the worker’s machine. The subtle point is that, in order to outsource the computation, both the pool operator and the worker may collaborate to prevent the worker from having the entire r in its memory, while still allowing it to perform the mining work. In particular, they may apply arbitrary strategies to split the computation in such a way that stealing the solution is not possible. This is similar to the situation in [16, 24], where, as described above, the malicious prover is interested in not having the entire input M in its memory, while still being to convince the verifier that it knows M .

While in this context, the MPCs and TEEs are not very relevant (do to the slowdown caused by MPCs and the inefficiency of TEEs for massive parallel hashing), other strategies may be applied to split the knowledge of certain strings between the worker and the operator. Hence, similarly, to what is done in [16, 24], the protocol of [30] is very sensitive to the way “knowing” a particular string is modeled. In particular, as we show in Sec. 3.3, the parties can exploit an iterative structure of the hash function to split the knowledge of r between them, in such a way that the worker can still perform the mining work, but cannot steal the solution.

1.1 Our contribution

The constructions in [30, 24] rely on hash functions modeled as random oracles [4]. Our key observation is that the modelling in used there is problematic if the hash function H is constructed in an *iterative* way, or in some other way that allows partial pre-computation. In particular, the construction of [30] is not secure when instantiated with several standard hash functions, and the construction of [24] is not as secure as claimed when instantiated with the Bitcoin mining hash function (double SHA-256). More concretely, our contribution is as follows.

- (A) We present a real-life attack on the nonoutsourcable puzzles protocol of [30] when instantiated with several popular hash functions (SHA-256 and SHA-512). The attack is based on exploiting the iterative structure of the hash function. We prove formally that this attack works. More concretely, it is based on the fact that the standard hash functions are typically constructed using iterative methods such as Merkle-Damgård or sponge paradigms (see Sec. 2.1). A hash function H constructed this way can be viewed as a mode of operation for the *iterative function* of fixed input length (called a *compression function* in the case of Merkle-Damgård and a *permutation function* in sponge). This means that if H is applied to a longer input M , then H can be pre-computed on M ’s prefix in such

a way that the result of this precomputation partially hides information about this prefix. We exploit this property to split the knowledge of certain strings between the pool operator and the worker.

- (B) We outline how the scheme of [30] could be modified to prevent the attack from point (A) in a way that would allow a security proof in the model of [16].
- (C) We observe a similar issue in the work by Kelkar et al. [24]. Specifically, we show that their protocol is vulnerable to an attack that exploits the hashing structure used in Bitcoin mining. This attack does not rely on the iterative structure of the hash function, but rather on the fact that the so-called *Bitcoin block header* consists of a field that is a Merkle root of the transactions included in the block, and can be precomputed separately. As above, we prove formally that this attack works. We also implement this attack on existing hardware. Our experiments show that the attack is feasible and relatively cheap, especially when the algorithm of [24] is used as a protection against the TEE-attacks, and is based on legacy Bitcoin mining hardware (see Table 2).
- (D) We outline provably secure countermeasures to the attack from point (C).

As of now, the protocols that we break remain academic proposals (both published at ACM CCS), and to our knowledge, they have not been deployed in real life. Our results serve as a warning label, illustrating the dangers of using the oracle model in a way that is not compatible with typical implementations of hash functions, in the context of modeling individual knowledge. Our work also illustrates the suitability of the adversarial model of [16] to study the security of protocols involving multiple mutually distrustful yet cooperating parties.

1.2 Related Work

In this section, we describe the related work, focusing first on the nonoutsourcable puzzles protocol and on the proofs of individual knowledge.

Nonoutsourcable puzzles. As mentioned above, the notion of nonoutsourcable puzzles was introduced by Miller et al. in [30]. After the publication of [30], several papers have been published that build on the idea of nonoutsourcable puzzles (e.g. [43, 42, 21]). The construction of [30] was also described in numerous survey papers [41, 37, 2] and books [32, 34], and it was also taught at a Coursera course on cryptography⁴. A heuristically-secure alternative construction of nonoutsourcable puzzles was proposed in [12]. An attack on the nonoutsourcable puzzles protocol of [30] was presented in [9]. It is, however, based on the assumption that the underlying blockchain supports the smart contracts, and requires the pool members to put large deposits aside (that would be confiscated if they steal the solution). In contrast, our attack does not require any smart contracts and concerns directly the primitive of nonoutsourcable puzzles.

Proofs of individual/complete knowledge. *Proofs of individual knowledge* were introduced in [16], and independently, under the name of *proofs of complete knowledge*, in [24]. As highlighted above, the goal of this primitive is to allow a prover to convince a verifier that it knows a message M with an additional guarantee that M must be known entirely by a single entity. The main difference between the two papers is that [16] is more focused on the theoretical aspects of the primitive, while [24] is more focused on the practical applications. The advantage of the approach of [16] is that M can be arbitrarily long, while the approach of [24] is limited to short messages. On the other hand, however, unlike [16], the approach of [24] is designed to be compatible with the existing hardware for massive hash computations, more concretely, the *ASICs* (Application-Specific Integrated Circuits) that are used in Bitcoin mining. Recently, papers that build on the notion of individual/complete knowledge have been published in [1, 18]

Other related work. The individual cryptography model was also used in the context of secret sharing scheme [17, 6] and threshold encryption [10]. Knowledge possession was also studied in the context of proofs of retrievability [7], proofs of replication [19], and proofs of space [15]. These primitives are, however, not

⁴ A course by Arvind Narayanan, see <https://www.coursera.org/learn/cryptocurrency>.

directly related to the individual knowledge, as they do not prevent the prover from splitting the message between multiple machines (see also [16, 24]). Weaknesses of the random oracle model were studied before, e.g., in [8, 26, 3, 25]. These works, however, do not consider practical attacks on the protocols for individual (or complete) knowledge. Moreover, this paper is concerned with a careful use of the random oracle model, rather than standard model constructions. On the theoretical side, a long and important line of work on *indifferentiability* [27] analyses random-oracle modelling of hash functions, taking into account their iterative structure. These papers, however, do not consider the individual knowledge model and, hence, are not directly related to our work.

1.3 Organization of the paper

The paper is organized as follows. In Sec. 2.2, we provide the necessary background and introduce some notation. In Sec. 3, we present our attack on the constructions of nonoutsourcable puzzles and outline countermeasures to it. In Sec. 4, we show an attack on the proof of complete knowledge of [24] and discuss possible countermeasures.

2 Preliminaries

We use standard cryptographic notation throughout this paper. The symbol $\leftarrow \$$ denotes uniform sampling from a set, i.e., for a finite set $\mathcal{X}$, we write $x \leftarrow \$ \mathcal{X}$ to indicate that x is chosen uniformly at random from $\mathcal{X}$. For a security parameter $\lambda \in \mathbb{N}$, we write $\text{negl}(\lambda)$ to denote a negligible function in λ . The symbol $\approx$ denotes computational indistinguishability between two probability distributions or random variables, meaning that no probabilistic polynomial-time algorithm can distinguish between them with non-negligible advantage.

2.1 Iterative hash functions

In our attack in Sec. 3 we strongly rely on iterative structure of the modern hash functions (i.e., the Merkle-Damgård [28, 13] or sponge [5] paradigms). In this approach, used in all popular hash functions, including all the widely-used SHA-family functions, the message is processed in fixed-size blocks, and the hash function is built by iterating a fixed-length *compression* or *permutation* function. Below, we briefly recall how the iterative hash functions are constructed from fixed-length iterating functions. Our presentation captures both the Merkle-Damgård and the sponge paradigms. To cover both paradigms, and to be able to discuss for which concrete hash functions our attacks apply (see Sec. 3.4), we need a relatively general definition of an iterative hash function and three parameters (denoted α , β , and λ). Let $h: \{0, 1\}^{\alpha+\beta} \rightarrow \{0, 1\}^\alpha$ be an *iterating function* (i.e., a compression [28, 13] or a permutation [5] function). We construct an *iterative* hash function $H: \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ from h as follows. Define recursively a function $G: (\{0, 1\}^\beta)^* \rightarrow \{0, 1\}^\alpha$ as (below each m_j is a binary string of length β):

- let $G(m_0) := m_0$,
- for $m = (m_1 \parallel \dots \parallel m_n)$, let $G(m) := h(G(m_0 \parallel \dots \parallel m_{n-1}) \parallel m_n)$.

Each value $G(m_0 \parallel \dots \parallel m_i)$ is called the (*intermediate*) *state* of the computation, and α is called the *state size*. Let $IV \in \{0, 1\}^\alpha$ be a fixed string called an *initialization vector*. The function H takes as input a binary string $m \in \{0, 1\}^*$ and then performs the following steps:

1. Pad m with a string $\text{padding}_{|m|}$ so that the resulting string $m' = m \parallel \text{padding}_{|m|}$ has length that is a multiple of β . This has to be done in a prefix-free way, e.g., by $\text{padding}_{|m|}$ having a form 10^* concatenated with a binary representation of the length of m in bits.
2. Write $m' = (m_1 \parallel \dots \parallel m_k)$, where each m_i has length β .
3. Output $\text{out}(G(IV \parallel m_1 \parallel \dots \parallel m_k))$, where out is some fixed function that maps $\{0, 1\}^\alpha$ to $\{0, 1\}^\lambda$.

Frequently, in the hash functions based on the Merkle-Damgård paradigm $\alpha = \lambda$, and `out` is the identity function, i.e., $\text{out}(x) = x$ for all $x \in \{0, 1\}^\lambda$. In case of the more modern sponge constructions, α does not need to be equal to λ , and `out` can be a much more complex function. The iterative structure permits to *precompute* the value of H on a prefix of a long input m . More precisely, consider a message m (of length greater than β), and write $m = m^0 \parallel m^1$, where m^0 is of length that is a multiple of β . Then the value of $H(m)$ can be computed as

$$H(m) = \text{postcomp}_H^{|m|}(\text{precomp}_H(m^0), m^1)$$

where

$$\text{precomp}_H(m^0) = G(IV \parallel m^0) \tag{1}$$

$$\text{postcomp}_H^{\text{len}}(x, m^1) = G(x \parallel m^1 \parallel \text{padding}_{\text{len}}) \tag{2}$$

(in the above, len is an integer denoting the length of m).

2.2 Merkle trees

Let us also recall the classic *Merkle tree* construction [29] that allows us to commit to a set of values and later prove the membership of a value in the committed set. Let L be a power of 2 and let $\lambda \in \mathbb{N}$ be some parameter. Let F denote a function $F : \{0, 1\}^{2\lambda} \rightarrow \{0, 1\}^\lambda$. Here F can be a hash function restricted to inputs of length 2λ , or simply a hash function compression function (see Sec. 2.1). We define $\text{Merkle}_F : (\{0, 1\}^\lambda)^L \rightarrow \{0, 1\}^\lambda$. On input a list of L values $\text{leaf}_1, \dots, \text{leaf}_L \in \{0, 1\}^\lambda$, the function Merkle_F outputs a labelled binary tree T (called a *Merkle tree*) with L leaves, where the label of the i th leaf is equal to leaf_i , and each internal node is labelled with the value of F applied to the concatenation of the labels of its two children. Moreover, let $\text{mroot}(T)$ denote the label of the root of Merkle tree T , and for $i = 1, \dots, L$, let $\text{MProof}(T, i)$ denote the Merkle proof for the i th leaf of a tree T , i.e., a branch of T going from the root to this leaf, including the leaf node itself, and the sibling for every node on the path from the leaf to the root. Merkle trees can also be easily updated in time logarithmic in the number of leaves. For a Merkle tree T as above, let $\text{update}(T, i, \text{leaf})$ denote the Merkle tree T' that is obtained from T by replacing the i th leaf with leaf and recomputing the tree. Note that for a fixed i th leaf, the value of $\text{mroot}(T)$ depends on the value of leaf_i and on the labels of the sibling nodes on the path from the i th leaf to the root (denote this sequence as $\text{siblings}(T, i)$). Therefore, given $\text{MProof}(T, i)$, the update operation can be performed in time $O(\log L)$.

3 Nonoutsourcable puzzles of [30] – attacks and countermeasures

We start this section by recalling the definition and construction of nonoutsourcable puzzles from [30]. Next, we present our attacks on the construction of [30] and propose our countermeasures. As explained in Sec. 1.2, the goal of the nonoutsourcable puzzles is to prevent the pool operator from outsourcing mining work to the pool members by ensuring that pool members can “steal” the solution to any puzzle they have solved. We now describe this notion in more detail, using the framework from [30], slightly adapted to our purposes. The puzzle systems are parametrized by a hash function, and the bulk of work in solving the puzzle consists of computing this hash function on different inputs (this work is called *mining*, and the party performing it is called a *miner*). To stay consistent with the rest of our paper, we model the *work time* in terms of the number of calls to the (underlying) hash iterating function h modelled as an oracle Ω_h ⁵. We will often use the symbol H , which, as described in Sec. 2.1, denotes the hash function constructed from the iterating function h .

⁵ Miller et al. model it in terms of abstract “units of time”, and assume that a single “puzzle scratch” takes a fixed amount of time.

3.1 The definition of Miller et al. [30]

Miller et al. [30] start with the following definition (see [30, Def. 1]). A *scratch-off puzzle* consists of three algorithms: $\mathcal{G}$, Work , and Verify . Algorithm $\mathcal{G}$ on input a security parameter 1^λ generates a *puzzle instance* $\text{puz} \in \{0, 1\}^*$ (or simply “puzzle”) of length λ . This is an unpredictable random string, e.g., the hash of the previous block in a blockchain, and $\mathcal{G}$ is an abstraction of a puzzle generation process. Algorithm Work takes a puzzle instance puz , a payload $m \in \{0, 1\}^*$, a parameter t that upper bounds the algorithm work time, and outputs a *ticket* if the worker was able to solve the puzzle using t calls to the oracle Ω_h (or $\perp$, otherwise). The probability that $\text{Work}(\text{puz}, m, t) \neq \perp$ depends on a difficulty parameter d and the work time parameter t . Finally, algorithm Verify takes as input a puzzle instance puz , a payload m , and a *ticket*, and outputs 1 if the *ticket* is *valid* for the puzzle instance and payload, or 0 otherwise.

We require that a scratch-off puzzle satisfy the following properties. First, it must be *correct*, meaning that for any puzzle instance puz , payload m , and $\text{ticket} := \text{Work}(\text{puz}, m, t)$ if $\text{ticket} \neq \perp$, then $\text{Verify}(\text{puz}, m, \text{ticket}) = 1$. Second, it must be ζ -*feasible*, meaning that algorithm Work with work time at most t , will output a ticket with probability at least ζ (where $\zeta \in [0, 1]$ is a function of t and d). Third, it has to be μ -*non-transferable*⁶ (for a constant $\mu \in [0, 1]$), meaning that for a random puzzle puz any adversary with work time at most $\mu \cdot t$ cannot output a valid (m, ticket) pair with probability greater than $\zeta(t, d) + \text{negl}(\lambda)$, even if the adversary can see a polynomial number of valid tickets on messages $m_1, \dots, m_n$ of her choice (where each $m_i \neq m$). This requirement is needed to prevent stealing the tickets once they are broadcast to the network (see [30, Def. 2] for the formal definition of non-transferability).

To define nonoutsourcability, Miller et al. [30] introduce the notion of an “outsourcing protocol”. Let t_W and t_{Op} be the number of oracle calls to Ω_h that the worker and the pool operator can make, respectively. Moreover, let t_E be an “effective time” parameter whose role is explained in a moment. Formally, a (t_W, t_{Op}, t_E) -*outsourcing protocol* for a scratch-off puzzle $(\mathcal{G}, \text{Work}, \text{Verify})$ is a pair of machines (W, Op) , where W is a *worker*, and Op is a *pool operator* such that the following holds. Consider a game in which the pool operator Op is given a puzzle instance $\text{puz} \leftarrow \mathcal{G}(1^\lambda)$, and the worker W is given no input. The parties interact, and at the end, the pool operator outputs a *ticket* and a payload m (or $\perp$). It is required that the *success probability*, i.e., the probability that the pool operator outputs a valid ticket given a puzzle instance puz , is at least $\zeta(d, t_E)$ (where ζ is the feasibility parameter of the scratch-off puzzle, see above). Informally, the parameter t_E captures the effective work time by the protocol participants (i.e., the amount of work time that a “solo miner” would need to work to achieve the same success probability as the outsourcing protocol). To ensure that the protocol is useful in practice, t_E should be much larger than the amount of oracle calls made by the pool operator t_{Op} . Of course, due to the μ -non-transferability requirement, $t_W + t_{Op}$ must be at least $\mu \cdot t_E$.

A scratch-off puzzle $(\mathcal{G}, \text{Work}, \text{Verify})$ is nonoutsourcable if there exists a malicious worker W^* that can steal the solution to the puzzle. Formally, Miller et al. [30] define the non-outsourcability of a scratch-off puzzle as follows. Miller et al. [30] define $(\mathcal{G}, \text{Work}, \text{Verify})$ to be (t_W, t_{Op}, t_E) -*nonoutsourcable*⁷ if for every outsourcing protocol (W, Op) with parameters t_E, t_{Op} and t_W , there exists a malicious worker W^* , working in time t_W , that can output a valid ticket for a random message m^* with probability at least $\zeta(d, t_E)$ (after interacting with Op on random puzzle instance $\text{puz}^* \leftarrow \mathcal{G}(1^\lambda)$). To prevent the pool operator from distinguishing between the honest and malicious worker, we require that the execution of the protocol (W^*, Op) is indistinguishable from the execution of (W, Op) (from the point of view of Op). More precisely, for every 1^λ , let $\text{view}^*(1^\lambda)$ denote the view of the pool operator Op in the execution of the protocol (W^*, Op) , and let $\text{view}(1^\lambda)$ denote the view of the pool operator in the execution of (W, Op) . Then, these two views must be computationally indistinguishable, i.e., $\text{view}^* \approx \text{view}$.

⁶ The authors of [30] also require that Work can be parallelized. We skip this requirement, as it is not relevant to our paper. They also state another requirement called “incompressibility” that is less general than non-transferability. For simplicity, we merge both requirements into a single one.

⁷ The definition in [30] has two additional parameters: α and p_S . We omit them, since they are not relevant to our paper.

3.2 The scheme of Miller et al. [30]

The scheme of Miller et al. [30] is depicted in Fig. 1, copied from [31], with only minor changes to the notation. The second difference is that we denote the number of scratch attempts by $\hat{t}$, instead of t as in [31]. This is done to avoid confusion with the work time parameter t used in our definition of outsourcing protocols⁸. It is chosen to be the maximal number such that the total number of hash function calls made by **Work** is at most t . The relation between $\hat{t}$ and t is discussed in Sec. 3.4. The scheme in Fig. 1 uses additional parameters L, d, q , and q' whose role is as follows. The parameter L is the number of leaves of a Merkle tree built by the **Work** algorithm, the parameter d sets the puzzle difficulty, the parameters q and q' are the numbers of leaves selected during computation of the **Work** algorithm in different steps of the procedure.

The algorithm **Work** consists of three main phases. In the first phase (“Construct Merkle Tree”, see Step 1), the worker sets up a Merkle tree with a root label *digest* from a list of leaves with randomly-selected values in the leaves. In the second (“Scratch”) phase (see Step 2), the worker essentially runs mining on the leaves of the Merkle tree to obtain a PoW solution. The mining is done by randomly trying different values $r \leftarrow_{\$} \{0, 1\}^\lambda$. The PoW solution is bound to the root of the Merkle tree *digest*, and the solution ticket includes a random value confirming the validity of the solution. This phase uses an **Encode** function that encodes a set of Merkle branches into a single binary string, which is used to represent the set of branches. The function **Encode** of [31] simply represents the set of branches as a concatenation of the branches (the concrete encoding of branches is not specified, and is irrelevant for our attack). In the third (“Sign payload”) phase (see Step 3), the worker signs an arbitrary message which is again bound to the *digest*. The ticket for the whole procedure includes both the PoW solution and the signature of the message.

The main idea of Miller et al. [30] is that in the scheme a valid ticket contains the PoW solution as well as the signature of the message, and both of these are bound to the same Merkle tree root *digest*. As the authors claim, it is infeasible to find (with high probability) a valid PoW solution without knowing the majority of the leaves of the Merkle tree and the random value r confirming the validity of the solution. At the same time, having the solution in hand also implies the possibility of signing an arbitrary message. Hence, seemingly the scheme is nonoutsourcable. The authors of [30] also provide a security proof for the scheme in the full version of their work [31]. Crucially, their proof assumes that the hash function H is modeled as a random oracle, even when applied to long messages. This appears in their proof in Definition 6 in [31], where the authors define “good scratch attempt” by referring to the random oracle calls taking as input the entire string. The rest of their proof relies on this definition, and hence on this assumption. In particular, it is implicitly assumed that if the oracle computed H on some input x , then either the worker or the operator must “know” x entirely. As highlighted before, this assumption is not satisfied in practice, as hash functions are typically constructed using iterative methods (see Sec. 2.1) such as Merkle-Damgard [28, 13] or sponge [5] paradigms. This key observation is the main starting point of our attack that we present in the next section.

3.3 Our attack

We are now ready to present our attack on the non-outsourcable puzzles of [30]. Following [30], we assume that the puzzle length equals λ . A detailed description of (W, Op) appears on Fig. 3. Let us start with its informal description. The original construction of [30] expects the worker W to mine over the nonce r . However, nothing prevents the parties from making r fixed, and letting the worker mine by sampling fresh leaves in the Merkle tree. Of course, each change of the leaves results in a fresh value of *digest*, and hence the value of $H(puz \parallel r \parallel \sigma_\eta)$ changes as well, so the probability of finding a valid solution remains the same in each scratch attempt. Crucially, for a large class of parameters (see Lemma 1), the worker can perform this mining without knowing the full nonce r . In our attack, the operator computes hashes involving r only in the form $H(puz \parallel r \parallel \cdot)$, where *puz* is the puzzle (see Steps (2b) and (2e)). The iterative construction of the hash function allows the operator Op to precompute the part depending on r with only a single query, so that the worker W can postcompute the rest of the hash. Since the knowledge of r is needed to produce

⁸ In [31] the “unit of work” is the scratch attempt, while in our paper it is the hash query.

- $\text{Work}(puz, m, t)$:
 1. **Construct Merkle tree.** Sample L random strings $leaf_1, \dots, leaf_L \leftarrow \{0, 1\}^\lambda$, and then construct a Merkle tree $T := \text{Merkle}_H(leaf_1, \dots, leaf_L)$ over the $leaf_i$'s using H as the hash function. Let $digest := \text{mroot}(T)$ denote the value in the root of this tree.
 2. **Scratch.** Repeat the following *scratch attempts* as long as the budget for hash queries allows for it, i.e., so that the total number of queries to H is at most t minus the number of queries needed in Steps 1 and 3 (denote the number of scratch attempts by $\hat{t}$, see also Sec. 3.4 for an estimation of $\hat{t}$ as a function of t):
 - (a) Draw a random nonce, $r \leftarrow \{0, 1\}^\lambda$.
 - (b) Compute $\eta := H(puz \parallel r \parallel digest)$, and use the value η to select q distinct leaves from the tree.
 - (c) Let $B_1, B_2, \dots, B_q$ denote the Merkle proofs (see Sec. 2.2) for the selected leaves.
 - (d) Compute $\sigma_\eta := \text{Encode}(B_1, \dots, B_q)$ where $B_1, \dots, B_q$ are in the sorted order.
 - (e) If $H(puz \parallel r \parallel \sigma_\eta) < 2^{\lambda-d}$ then record the solution pair (r, σ_η) and goto “Sign payload”.
 3. **Sign payload.** If no solution was found, return $\perp$. Otherwise, sign the payload m as follows:
 - (f) Compute $\eta' := H(puz \parallel m \parallel digest)$, and use the value η' to select a set of $4q'$ distinct leaves from the tree such that these leaves are not contained in σ_η . From these, choose an arbitrary subset of q' distinct leaves. Collect the corresponding branches for these q' leaves, denoted $B_1, B_2, \dots, B_{q'}$.
 - (g) Let $\sigma'_\eta := \text{Encode}(B_1, \dots, B_{q'})$ in sorted order of i .
 4. Return $\text{ticket} := (digest, r, \sigma_\eta, \sigma'_\eta)$.
- $\text{Verify}(puz, m, \text{ticket})$:
 1. Parse $\text{ticket} := (digest, r, \sigma_\eta, \sigma'_\eta)$.
 2. Compute $\eta := H(puz \parallel r \parallel digest)$ and $\eta' := H(puz \parallel m \parallel digest)$.
 3. Verify that σ_η and σ'_η contain leaves selected by η and η' respectively.
 4. Verify that σ_η and σ'_η contain valid Merkle paths with respect to $digest$.
 5. Verify that $H(puz \parallel r \parallel \sigma_\eta) < 2^{\lambda-d}$.

Fig. 1: The original scheme of [30] from the extended version of this paper [31].

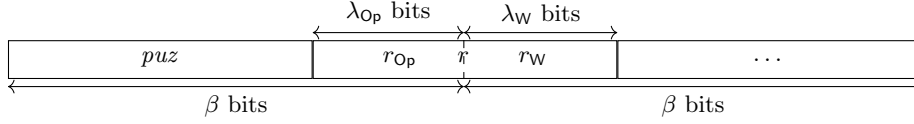


Fig. 2: The depiction of λ_W as the part of the randomness λ and the puzzle puz that falls into the last block of the hash input, and is under control of the worker.

the ticket, this prevents the worker from stealing it. Hence, the mining can be outsourced. For the detailed argument, see the proof of Lemma 1 below.

Lemma 1. Assume that the hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ is constructed iteratively using a iterating function $h : \{0, 1\}^{\lambda+\beta} \rightarrow \{0, 1\}^\lambda$. Consider the scheme from Fig. 1 and assume that

- (A1) $\lambda \leq \beta$
- (A2) $2\lambda \geq \beta$, and
- (A3) $2^{\lambda-\beta} = \text{negl}(\lambda)$,

where β is the block size of the hash function and λ denotes a security parameter. Then, for every t , there exists a (t_W, t_{Op}, t) -outsourcing protocol (W, Op) in with $t_{Op} = 1$ and $t_W = t + \hat{t} \cdot \log_2 L$, where $\hat{t}$ is the number of scratch attempts in Step 2 of the algorithm Work (see also Sec. 3.4 for an estimation of $\hat{t}$ as a function of t).

Before proceeding with the proof of the lemma, let us state its following implication (for its derivation, see Sec. 3.4), which shows that the original scheme of [30] is not nonoutsourcable even for a very minimal parameter choice (the “effective work” is essentially equal to the work of the worker alone).

Corollary 1 (informal). *Under the assumptions of Lemma 1, the scheme from Fig. 1 is not $(t_W, 1, t)$ -nonoutsourcable for $t_W \approx t$.*

Due to the assumptions of Lemma 1, the attack does not work for every possible choice of H . However, as we argue below, it works if H is any of the following hash functions: SHA-256, SHA-512, and, in a slightly weaker sense, SHA3-512 (see, e.g., [38]). Table 1 summarizes the relevant parameters of these functions. It can be easily seen that Assumptions A1 and A2 of the lemma are satisfied for all three functions. Assumption A3 is

	λ	β	$\lambda - \beta$
SHA-256	256	512	-256
SHA-512	512	1024	-512
SHA3-512	512	576	-64

Table 1: Comparison of SHA-256, SHA-512, and SHA3-512: security parameter λ (equal to the hash output size), block size β , and the difference $\lambda - \beta$.

satisfied in practice for SHA-256 and SHA-512 with a huge margin. For SHA3-512 it is on the edge, as in this case $\lambda - \beta = -64$, and 2^{-64} may not be “negligible enough” for some very powerful adversaries (although it is hard to imagine malicious workers who would be interested in investing so much effort in stealing the puzzle solution).

Proof (of Lemma 1). An informal description of the outsourcing protocol appears at the beginning of Sec. 3.3. Going more into details, the outsourcing protocol (W, Op) works as follows (see also Fig. 2). The operator samples the full nonce and splits r into two parts: r_{Op} and r_W . The operator then precomputes $x := \text{precomp}_H(puz \parallel r_{Op})$. Note that due to Assumption A2 the length of $puz \parallel r$ is at least as large as the block length β , so this precomputation is possible. The operator then sends the precomputed value x and the worker’s portion r_W to the worker W . Crucially, the precomputation x hides the operator’s secret r_{Op} while still enabling the worker to evaluate hash functions of the form $H(puz \parallel r \parallel \cdot)$ using postcomputation. Hence, in the “Sign payload” phase (Step 3), the worker cannot compute the ticket, as she does not know the entire r (cf. Step 3). Of course, this only works if the “ r_{Op} ” part of the nonce is long enough to be infeasible for the worker to guess it, if chosen randomly. Let $\lambda_W := |r_W|$ and $\lambda_{Op} := |r_{Op}|$. We now have the following:

$$\lambda_{Op} = \lambda_{Op} + \lambda_W - \lambda_W \tag{3}$$

$$= \lambda_{Op} + \lambda_W - \lambda_W - (\lambda + \lambda_{Op}) \pmod{\beta} \tag{4}$$

$$= -\lambda \pmod{\beta}, \tag{5}$$

Where in Eqs. (3) and (5) we applied simple addition and subtraction, and in Eq. (4) we used the fact that $\lambda + \lambda_{Op}$ is a multiple of β . Now, by Assumption A1, Eq. (5) implies that $\lambda_{Op} = \beta - \lambda$. Finally, by Assumption A3, we have that $2^{-\lambda_{Op}} = 2^{\lambda - \beta} = \text{negl}(\lambda)$. This implies that the worker cannot guess r_{Op} except with negligible probability.

Since knowing r is required to produce a valid ticket in the “sign payload” phase that produces σ'_η , the worker cannot therefore construct a valid ticket on its own, while it is still able to do meaningful work for the pool operator. Only the operator, who knows the complete nonce $r = r_{Op} \parallel r_W$, can perform this final step and generate the complete ticket $(digest, r, \sigma_\eta, \sigma'_\eta)$. This prevents solution theft while enabling computational outsourcing. Let us now analyze the work time of the operator and the worker. The operator Op makes only a single query to the hash function h to perform the precomputation $x := \text{precomp}_G(puz \parallel r_{Op})$, which takes one query to the iterating function h . To estimate the work time of the worker, observe (see Fig. 3) that we let the worker sample $L - 1$ leaves *once*, and then replace the last leaf with a fresh one. We do it for efficiency reasons, as changing only one leaf allows for computation of *digest* in $\log_2 L$ oracle calls by simply

- The operator **Op** takes as input a puzzle puz and a random message m and performs the following steps:
 1. Sample random $r = (r_{Op} \parallel r_W) \leftarrow \{0, 1\}^\lambda$, where $|r_{Op}| = \lambda_{Op}$ and $|r_W| = \lambda_W$.
 2. Perform the precomputation of h on $(puz \parallel r_{Op})$ (see Eq. (1)), i.e., let $x := \text{precomp}_G(puz \parallel r_{Op})$.
 3. Send (x, r_W) to the worker **W**.
- The worker **W** receives (x, r_W) , computes $r := r_{Op} \parallel r_W$ and performs the following steps
 - **Construct Merkle tree.** Sample $L - 1$ random strings $leaf_1, \dots, leaf_{L-1} \leftarrow \{0, 1\}^\lambda$, and then construct a Merkle tree $T := \text{Merkle}_H(leaf_1, \dots, leaf_{L-1}, 0^\lambda)$.
 - **Scratch.** Repeat the following scratch attempt $\hat{t}$ times (where $\hat{t}$ is as in the original protocol, see Step 2b of Fig. 1):
 1. Draw a random leaf $leaf_L \leftarrow \{0, 1\}^\lambda$ and update the Merkle tree T by replacing the last leaf with $leaf_L$, i.e., let $T' := \text{update}(T, L, leaf_L)$ (see Sec. 2.2). Let $digest := \text{mroot}(T')$.
 2. Compute $\eta := H(puz \parallel r \parallel digest)$, by postcomputing (see Eq. (2)) the value x received from the operator **Op**, i.e., let $\eta := \text{postcomp}_h^{3\lambda}(x, r_W \parallel digest)$ (“ 3λ ” corresponds to the length of $puz \parallel r \parallel digest$) and use the value η to select q distinct leaves from the tree exactly as in Step (2b) of the original scheme
 3. Let $B_1, B_2, \dots, B_q$ denote the Merkle proofs for the selected leaves (*this is identical to Step (2c) of the original scheme*).
 4. Compute $\sigma_\eta := \text{Encode}(B_1, \dots, B_q)$ in sorted order of i (*this is identical to Step (2d) of the original scheme*).
 5. Compute $y = H(puz \parallel r \parallel \sigma_\eta) < 2^{\lambda-d}$ by postcomputing the value x received from the operator **Op**, i.e., let $y := \text{postcomp}_H^{len}(x, r_W \parallel \sigma_\eta)$ (here, “ len ” denotes the length of $puz \parallel r \parallel \sigma_\eta$). If $y < 2^{\lambda-d}$ then send $(leaf_1, \dots, leaf_L)$ to the operator **Op**.
- The pool operator **Op** receives $(leaf_1, \dots, leaf_L)$ from the worker **W** and performs the following steps:
 1. Verify that computation was done correctly by the worker **W**, i.e. that the leaves $(leaf_1, \dots, leaf_L)$ build up a Merkle tree for the $digest$ and that $\eta = H(puz, r, digest)$ determines the set σ_η and that $H(puz, r, \sigma_\eta) < 2^{\lambda-d}$.
 2. Perform the “sign payload” step of the original scheme (Steps (3f) and (3g) on Fig. 1) on message m to obtain σ'_η .
 3. Return $ticket := (digest, r, \sigma_\eta, \sigma'_\eta)$.

Fig. 3: Our outsourcing procedure for the scheme of Miller et al. Above, hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ is constructed iteratively using a iterating function $h : \{0, 1\}^{\lambda+\beta} \rightarrow \{0, 1\}^\lambda$. For the discussion on the parameters, see Sec. 3

recomputing one path with the **update** procedure (see Sec. 2.2). Since we have $\hat{t}$ scratch attempts, the total overhead is $\hat{t} \cdot \log_2 L$. Resulting in $t_W = t + \hat{t} \cdot \log_2 L$ hash queries. This finishes the proof. $\square$

We also note that in practice, the solution of [30] still provides a certain degree of protection against outsourcing. This is because the malicious worker **W** in our attack can still steal the solution *after* she learns the operator’s part r_{Op} of the nonce. This would be, in particular, the case if the operator **Op** posts the solution publicly by announcing a new block on the blockchain.

3.4 Informal proof of Corollary 1

We will now show that in our attack, the worker’s work time is only slightly larger than the work time t of a solo miner. Let us first estimate the value of the parameter $\hat{t}$ as a function of the work time parameter t . This is needed, since, unlike in the original paper [30], we count the work time as the number of calls to the hash iterating function h (see Sec. 2.1), rather than the number of “scratch attempts” (i.e., the number of iterations of the loop in Step 2 of the algorithm **Work**). Let us first estimate the number of calls to the hash iterating function h made by the algorithm **Work** in Fig. 1, as a function of $\hat{t}$. We will use this estimation later to determine the value of the parameter $\hat{t}$ as a function of the work time parameter t . Constructing a Merkle tree with L leaves (in Step 1) requires $L - 1$ calls to H . Since each such call requires at most two

calls to h , constructing the Merkle tree requires at most

$$2(L - 1) \quad (6)$$

calls to h . Each scratch attempt requires: (1) recomputing η (one call to H , and hence to h) and (2) computing $H(puz \parallel r \parallel \sigma_\eta)$, which takes approximately $q \cdot \log_2 L$ calls to h (since σ_η contains q Merkle branches, each of length $\log_2 L$). Since we have $\hat{t}$ scratch attempts, the total number of calls to h made by the algorithm **Work** in the “Scratch” phase (Step 2) is approximately

$$\hat{t} \cdot (1 + q \cdot \log_2 L). \quad (7)$$

Finally, the “Sign payload” phase (in Step 3) requires approximately one call to H (and hence two calls to h , assuming that $|m| = \lambda$) for computing η' . Summing this with (6) and (7), we conclude that the total number of calls to h made by **Work** is approximately

$$2(L - 1) + \hat{t} \cdot (1 + q \cdot \log_2 L) + 2 = 2L + \hat{t} \cdot (1 + q \cdot \log_2 L). \quad (8)$$

To ensure that the total number of calls to h is at most t , we set $\hat{t}$ to be the maximal integer such that $2L + \hat{t} \cdot (1 + q \cdot \log_2 L) \leq t$, which gives

$$\hat{t} \approx \left\lfloor \frac{t - 2L}{1 + q \cdot \log_2 L} \right\rfloor. \quad (9)$$

Recall (cf. Lemma 1) that the work t_W of the worker in the outsourcing protocol is equal to $t + \hat{t} \cdot \log_2 L$. Since L is typically much smaller than t , and $q \gg 1$ (in [31] it is set to 10) thus, by Eq. (9), t_W is approximately equal to t .

3.5 Countermeasures

The attack presented above strongly relies on the parameters of the original scheme, and in particular on the fact that the authors of [30] decided to use a single parameter λ to denote the puzzle length, the randomness length, and the hash function output length. In the extended version of this paper, we show that this attack can be avoided if one is more careful with the parameter choice. Our main idea is to make sure that the input to the hash function of the Step 2b of the Scratch subroutine fits into a single block of the hash function. Then, one can replace the iterative hash function H with the underlying compression function h , whose evaluation cannot be so easily divided between the worker and the operator.

More precisely, let H be an Merkle-Damgård hash function with the block size β . Assume β is divisible by 4 (which is the case for all popular hash functions, see, e.g., [38]). Our scheme is essentially the same as the original one (see Fig. 1), except that we require the following:

1. the puzzle puz and randomness r are $\lambda = \beta/4$ bits long,
2. the hash function used to compute $digest$ has output length at most $\beta/2$, and
3. in Step 2b of the Scratch subroutine, the call to the hash function $H(puz \parallel r \parallel digest)$ is replaced with a call to the underlying compression function $h(puz \parallel r \parallel digest)$.

This means that $(puz \parallel r \parallel digest)$ has exactly β bits and fits into a single compression function. Requirement 1 is easy to satisfy in practice: in all real-life hash functions (see, e.g., [38]) $\beta \geq 512$, and hence $2^{-\lambda} \leq 2^{-128}$, which is negligible for all practical applications. Requirement 2 is also commonly satisfied: several popular hash functions (e.g., SHA-256) have output length equal to $\beta/2$. Alternatively, one can use a hash function with output length greater than $\beta/2$, but truncate its output to $\beta/2$. Note that for all real-life hash functions, $\beta/2 \geq 256$; hence, the truncation will not significantly affect the security of the hash function, also in the context of birthday-type attacks (see, e.g., [23]).

The intuition is that, in the absence of slow queries, the value η will be computed entirely either by the worker or by the operator. In the former case, the worker is able to steal the solution. The latter involves heavy

communication, as then the operator needs to receive the *digest* and send back the value η for each scratch attempt. To model the non-outsourceability of the modified scheme, we use the individual cryptography model of [17], which models the compression function h (not the entire hash function H) as a random oracle. Additionally, we need to assume a bound on the communication between the worker and the operator. This is because, in the absence of such a bound, the operator and the worker can simply split the set of leaves of the Merkle tree between themselves and compute η jointly by exchanging the messages for each scratch attempt. The proof will be provided in the extended version of this paper.

4 Proofs of Complete Knowledge of [24] – an attack and a countermeasure

As already highlighted in Sec. 1, the aim of proofs of complete knowledge (PoCK) of [24] is to ensure that a prover *actually knows* a witness w - as opposed to w being divided between a group of parties, secret-shared, or encumbered in a TEE enclave. This is captured by the notion of *forced revelation*: after a successful interaction with the verifier, the prover should be able to extract the witness w given only their own execution trace. Note that this extraction should be a process that the prover could actually perform, and should *not* be an abstract cryptographic extractor built on thought-experiments, such as forking or rewinding.⁹ The authors of [24] focus on achieving *zero-knowledge* PoCK (ZKPoCK), in which the interaction with the verifier does not leak any information about the witness w .

4.1 The original ASIC-ZKPoCK scheme

The key technique used by PoCK to achieve forced revelation is to require that the proving process involve interaction with external hardware, which can be eavesdropped on by the prover. The authors of [24] present two variants of their scheme. In our paper, we are interested in the variant called ASIC-ZKPoCK, which uses an ASIC (Application-Specific Integrated Circuit) as the external hardware. In the ASIC-ZKPoCK scheme of [24], this external hardware is a Bitcoin mining rig which performs a brute-force search to find values whose hash starts with many zeros. The *hashrate* (i.e., the per-second rate at which the hashes are computed) achieved by the mining rigs far exceeds that achieved by standard hardware. Hence, the only feasible way to find a valid proof within some timeframe τ is to leverage the mining rig’s capabilities. Therefore, this computation cannot be performed by an MPC protocol or a TEE enclave.

Since the authors of [24] are interested in constructing *zero-knowledge* PoCKs, they build their construction on top of a *sigma protocol*. Recall that a sigma protocol (see, e.g., [22]) is a public-coin 3-round zero-knowledge protocol between a prover and a verifier. More formally, let $\mathcal{R} \subseteq \{0, 1\}^* \times \{0, 1\}^*$ be a binary relation. The prover P^Σ in a sigma protocol consists of two algorithms P_1^Σ and P_2^Σ :

- $P_1^\Sigma(x, w) \rightarrow (a, st)$: takes as input the *statement* x and the *witness* w (such that $(x, w) \in \mathcal{R}$) and outputs the first message a of the protocol and some state st ,
- $P_2^\Sigma(st, c, x, w) \rightarrow s$: takes as input the state st , a random challenge $c \leftarrow \mathcal{C}$ (where $\mathcal{C}$ is the *challenge space*) from the verifier and outputs the final message s .

The verifier $V^\Sigma(x, (a, c, s))$ takes as input the statement x and the sigma protocol transcript (a, c, s) and outputs **true** if the transcript is *valid* for the statement x (and **false**, otherwise). Every sigma protocol satisfies completeness and special honest-verifier zero-knowledge properties (see, e.g., [22] for details). The property of sigma protocols that is particularly important for us is *special soundness*: given two accepting transcripts (a, c, s) and (a, c', s') with the same first message a but different challenges $c \neq c'$, it is possible to efficiently compute (“extract”) the witness w such that $(x, w) \in \mathcal{R}$. This property is exploited by the authors of [24] to achieve the forced revelation. On a high level, their idea is to design a scheme in which the prover P must use the mining rig to compute the sigma protocol responses for the same a and at least two different challenges c and c' . Since they assume that the prover P can eavesdrop on all communication with the mining rig, in

⁹ For example, the well-known knowledge extractor for Schnorr’s protocol requires “rewinding” of the machine of the prover.

this way they make the prover P learn two accepting transcripts with the same first message a but different challenges $c \neq c'$. Consequently, the prover P can use special soundness to extract the witness w .

A classic example of a sigma protocol is Schnorr's identification protocol [35]. In this case, the statement x is a public key g^w for some generator g of a group $\mathcal{G}$ of prime order q , and the witness w is the corresponding secret key. The prover's first message a is a commitment of the form $a = g^u$ for some random $u \leftarrow \mathbb{Z}_q$, and the prover stores $st = u$. The verifier's challenge c is an integer from $\mathcal{C} = \mathbb{Z}_q$, and the prover's final message is $s = u + c \cdot w \pmod{q}$. The verifier checks the validity of the transcript (a, c, s) by verifying that $g^s = a \cdot x^c$.

The construction of [24] works generically for any sigma protocol¹⁰ and is depicted in Fig. 5. We describe it in more detail a bit later. For a moment, let us only highlight that the prover P and the verifier V interact in n sequential, timed rounds. In each round i , the prover runs P_1^Σ to compute (a, s) , sends a to the verifier, and retains the state s . The verifier responds with a random nonce r and waits for a response π (encoding the output of P_2^Σ) within a fixed time limit τ . If the prover fails to respond in time, the verifier rejects. Otherwise, the verifier decodes the sigma protocol response s_i from π and checks its validity by verifying that $V^\Sigma(a, c_i, s_i) = \text{true}$. Finally, the verifier accepts only if at least y out of n responses π satisfy the proof-of-work condition, i.e., their hash values are below the required threshold. The requirement that it suffices to have y out of n valid responses comes from the fact that Bitcoin mining is a probabilistic process, and there is no guarantee that a valid proof-of-work can always be found within time τ .

Mining rig design. Let us first recall the relevant elements of the structure of a Bitcoin block header and the mining procedure (it is illustrated in Fig. 4). The *block header* is a result $(B \parallel \nu)$ of a concatenation of a *block header prefix* $B \in \{0, 1\}^{608}$ and a *nonce* $\nu \in \{0, 1\}^{32}$. The block header prefix B contains several fields. The one that is of particular importance for us is a field called the *Merkle root*, denoted $B.mroot$. It is a 256-bit string that is the root of a Merkle tree built over the sequence of hashes of Bitcoin transactions included in the block (see Sec. 2.2 for details on Merkle trees). The Merkle tree is computed using a SHA-256 hash function [39]. SHA-256 is an iterative, Merkle-Damgård hash function built using a compression function $H: \{0, 1\}^{256+512} \rightarrow \{0, 1\}^{256}$. For brevity, we will denote SHA-256 as H . Also, each transaction (before being included in the Merkle tree) is hashed twice using SHA-256.

Historically, to mine a new block, the miner would take a fixed block header prefix B and search for a nonce value ν such that the block header $(B \parallel \nu)$ double-hashes to a value below a specific target. More precisely, the miner searched for a nonce ν such that $H(H(B \parallel \nu)) < 2^{256-d}$, where d is a *difficulty parameter*. Unfortunately, the nonce field is only 32 bits, providing just 2^{32} possible values, which is insufficient for modern mining operations. For this reason, the mining process has evolved to allow modification of other parts of the block header. Technically, this is done by placing another nonce, called the *extranonce*¹¹, in the so-called *coinbase transaction cbase*. This transaction (in Bitcoin, used for paying the reward for the miner) is the first leaf of the Merkle tree that is used to compute the Merkle root $B.mroot$. To avoid changing the semantics of *cbase*, the extranonce is placed in one of the redundant fields of *cbase*. Let $cbase[\text{extranonce} := \xi]$ denote the coinbase transaction with extranonce ξ . We will write $cbase[\text{extranonce} := \perp]$ to denote *cbase* with the extranonce set to some default value (say: $\text{extranonce} := (0, \dots, 0)$). Bitcoin mining is predominantly implemented in silicon as an ASIC (Application-Specific Integrated Circuit). Therefore, it is useful to identify the data needed by the ASIC to compute a B with a given extranonce ξ . Note that B is a function of:

1. all the fields of the block header prefix B except the Merkle root $B.mroot$, and
2. the information needed to compute the Merkle root $B.mroot$ with the extranonce ξ placed in the coinbase transaction *cbase*. Note that the Merkle root $B.mroot$ can be computed from
 - (a) the extranonce ξ ,
 - (b) the rest of the coinbase transaction $cbase[\text{extranonce} := \perp]$, and
 - (c) the siblings on the Merkle branch from the root to the first leaf (i.e. $siblings(T, 1)$, see Sec. 2.2)

¹⁰ Technically, their ASIC-ZKPoCK requires that the final message s output by P_2^Σ is *computationally unique* given a and c . This is, however, irrelevant to our attack.

¹¹ Technically, the extranonce is broken into *extranonce1* and *extranonce2*, but this is irrelevant to this discussion.

All the data above, except the extranonce ξ (i.e. data in Points 1, 2b, and 2c), is sent to the ASIC in a *mining context* C . Let $C[\text{enonce} := \xi]$ denote the block header prefix obtained from C by computing the Merkle root with from the coinbase transaction $\text{cbase}[\text{enonce} := \xi]$ and $\text{siblings}(T, 1)$. To summarize: to mine a new block, the miner first constructs a context C and sends it to the ASIC. The ASIC searches for a pair (ν, ξ) such that the block header $(C[\text{enonce} := \xi], \nu)$ hashes to a value below the target d .

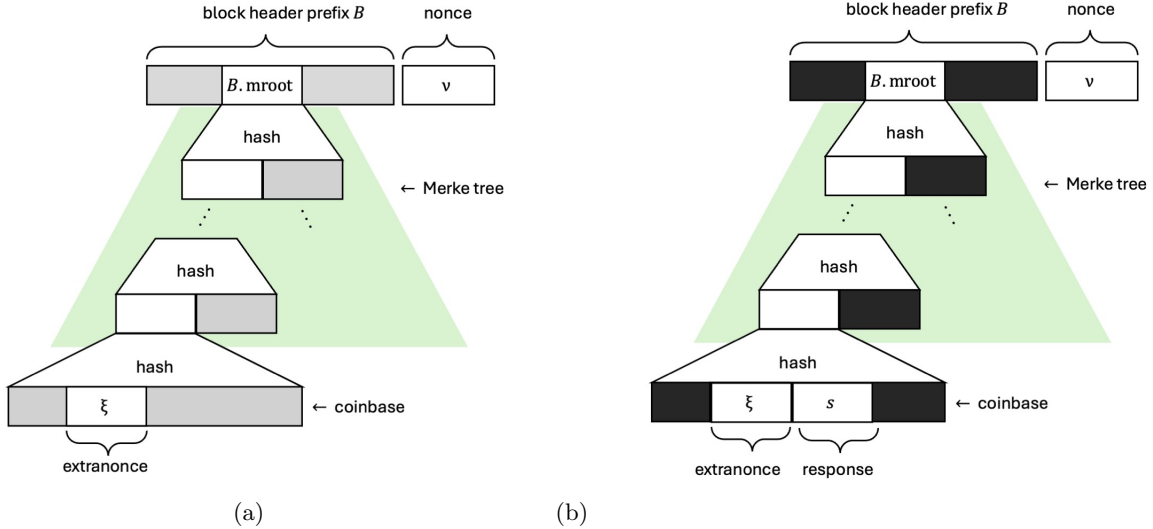


Fig. 4: An illustration of the Bitcoin block header structure. (a) The context C (sent to the ASIC miner) is indicated in gray; (b) the augmented context $\hat{C}$ is shown in black.

Implementation details of the ASIC-ZKPoCK scheme of [24]. We now describe the ASIC-ZKPoCK scheme of [24] (depicted in detail in Fig. 5). Recall that in this protocol, the prover P interacts with the verifier V to prove in zero-knowledge the knowledge of a witness w for a statement x for a relation $\mathcal{R}$. The prover P has access to an ASIC mining rig that performs Bitcoin mining. As explained above, the protocol is supposed to be designed in such a way that every prover (even a malicious one) has to be able to reconstruct w just by eavesdropping on its own communication with the mining rig. The protocol is built on top of a sigma protocol $P = (P^\Sigma, V^\Sigma)$ for $\mathcal{R}$ with challenge space $\mathcal{C}$ (e.g., $\mathcal{C} = \mathbb{Z}_q$ in case of the Schnorr protocol). Let $F : \{0, 1\}^* \rightarrow \mathcal{C}$ be a hash function modeled as a random oracle.

The main idea of [24] is to encode the sigma-protocol challenge-response pair (c, s) into a block header template that the prover passes to the ASIC mining rig. On the technical level, this is done as follows. We extend the concept of the context C used in Bitcoin mining to permit encoding of the sigma-protocol response s . This is done similarly to how the extranonce ξ is encoded into B , i.e., it is placed in the coinbase transaction, which, as a consequence, modifies the Merkle root. Let $\hat{C}$ denote this augmented context. Let $C_s := \hat{C}[\text{response} := s]$ denote the context $\hat{C}$ with the sigma-protocol response s encoded in it.

The prover P samples the augmented context $\hat{C}$ randomly (the protocol works regardless of how $\hat{C}$ is sampled, as long it is unlikely to repeat). Then, the prover computes the challenge $c := F(r || \hat{C})$ (where r is a random nonce sampled by the verifier, needed to avoid precomputation attacks). The prover then computes the sigma-protocol response $s := P_2^\Sigma(st, c, x, w)$ and places it in the augmented context $\hat{C}$ to obtain context $C_s := \hat{C}[\text{response} := s]$. The prover P then sends C_s to the ASIC mining rig. The mining rig then looks for a nonce pair $(\xi^*, \nu^*) \in \mathcal{S}$ (where $\mathcal{S}$ is a set of *allowed* nonce pairs $\mathcal{S}$ whose role will be explained later)

that solves the proof-of-work for the context C_s . If the ASIC finds such a nonce pair (ν, ξ) , then the prover P sends it together with C_s and s to the verifier. The verifier then checks if all the work performed by the prover is valid. This check, in particular, involves recovering c from C_s . Note that for a fixed a , the verifier learns only one pair (c, s) . Hence, by the ZK property of the sigma protocol, the verifier learns nothing about the witness w .

To ensure that the prover can only compute a response using a mining rig, the time the prover is given to find such a nonce pair (ν, ξ) is limited. This limit is denoted as τ . The set $\mathcal{S}$ of allowed nonce pairs and the difficulty parameter d are chosen in such a way that it is that a valid nonce pair (ν, ξ) is (1) likely to be found in time τ when ASIC is used for the search, but (2) *unlikely* to be found for a the first context C_s that the prover tries.¹² Therefore, to find a valid proof-of-work within time τ , the prover P must try many different contexts C_s (with the same response s). This, however, requires sending many different sigma-protocol responses s for the same first message a but different challenges c (since c is derived from C_s). Crucially, the authors of [24] assume that the only way the prover can use the ASIC mining rig to find a valid nonce pair (ν, ξ) is by sending it the context C_s (as we later show in Sec. 4.2, this assumption does not hold in practice; this observation is the basis of our attack). Therefore, by eavesdropping on its own communication with the ASIC mining rig, the prover P learns many different valid sigma-protocol responses s for the same first message a but different challenges c . Hence, it can compute the witness w using special soundness.

Since with a small but noticeable probability the ASIC mining rig may find a valid nonce pair (ν, ξ) for the first context C_s that the right receives, the protocol is repeated for n sequential rounds to ensure that with high probability at least two different contexts C_s are sent to the ASIC mining rig. To tolerate the fact that the prover may not find a valid nonce pair (ν, ξ) within time τ , we tolerate that only y out of n rounds need to be successful.

4.2 Our attack

We now show how MPC/TEE-based computation can be used in an adversarial way to break the forced revelation guarantee of the ASIC-ZKPoCK protocol. To this end, we construct a malicious prover P^* who proves knowledge of a witness w for a statement x , even though he does not *individually* (or “completely”) know w . This could be the case if w is shared by P^* with other parties using secret sharing, and P^* only holds a share of w . Alternatively, P^* could hold w hidden in a TEE enclave. In both cases, P^* can compute functions f (relative to a hash function H) of w using secure multiparty computation (MPC) or TEE-based computation. Following the conventions from the *individual cryptography*, any hash evaluation by f will be called *slow*.¹³ Of course, we need to assume that if computing f involves hashing, then it cannot be performed at the same speed as the mining rig. In our attack, the number of slow hash evaluations is small.

The main idea of our attack (for the detailed description, see Fig. 6) is similar to the attack on the nonoutsourcable puzzle from Sec. 3.3, since it exploits the fact that some values that are hashed together can be computed in two steps, hiding some parts of the input from the entity that sees that intermediate hash value only. As highlighted above, in the security argument of [24], the authors crucially rely on the assumption that the mining rig can only be used in a black-box way to compute the hash of the *entire* mining context C_s . Particularly, they assume that the communication between the prover and the mining rig involves sending s . However, since s is placed in a leaf of a Merkle tree, the prover can do the following:

1. Using slow hash evaluations, compute the Merkle root z of the tree containing s ,
2. Place z in the block header prefix, resulting in the block header prefix B , and
3. Send only B to the ASIC.

¹² More precisely, to guarantee (1) the parameter d has to be calibrated to the hashrate of the ASIC, while to guarantee (2) the set $\mathcal{S}$ has to be relatively small.

¹³ More precisely, in [16, 17, 6] the authors model the fact that a small number of hashes can be computed without any machine knowing their inputs (using MPCs or TEEs) by assuming that the adversary has a limited budget of so-called *slow oracle queries* that can be computed on shared inputs.

1. For each round $i \in \{1, \dots, n\}$:
 - (a) The prover samples $(a, st) \leftarrow \mathcal{P}_1^\Sigma(x, w)$ and sends a to V .
 - (b) The verifier samples a nonce r and sends it to P .
 - (c) *Prover P*:
 - i. Choose $\hat{C}$ randomly.
 - ii. Let $c := F(r \| \hat{C})$.
 - iii. Compute the sigma-response $s := \mathcal{P}_2^\Sigma(st, c, x, w)$.
 - iv. Send to the ASIC the context $C_s := \hat{C}[\text{response} := s]$.
 - (d) *ASIC*:
 - i. Look for a nonce pair $(\xi^*, \nu^*) \in \mathcal{S}$ such that $H(H(C_s[\text{nonce} := \xi^*] \| \nu^*)) < 2^{256-d}$.
 - ii. If you find such (ξ^*, ν^*) then return it; otherwise return “no hit”.
 - (e) *Prover P*:
 - i. If the ASIC returned a hit (ξ^*, ν^*) , send the *round solution* $(\hat{C}, s, \xi^*, \nu^*)$ to the verifier and proceed to the next round.
 - ii. Else:
 - A. If the deadline τ is approaching, then finish the round without submitting a proof.
 - B. Otherwise go back to the step where you sample a fresh C_s (Step 1c).
2. *Verifier V*: For each round solution $(\hat{C}, s, \xi^*, \nu^*)$ received from the prover in Step 1(e)i, perform the following checks:
 - (a) Check if it arrived before the round’s time limit τ .
 - (b) Check if $(\xi^*, \nu^*) \in \mathcal{S}$ and if the solution meets difficulty d , i.e, if $H(H(C_s[\text{nonce} := \xi^*] \| \nu^*)) < 2^{256-d}$, where $C_s := \hat{C}[\text{response} := s]$.
 - (c) Recompute $c := F(r \| \hat{C})$ from the received $\hat{C}$.
 - (d) Check if $V^\Sigma(x, (a, c, s)) = \text{true}$.

Accept iff *at least* y rounds pass all checks.

Fig. 5: The ASIC-ZKPoCK scheme of [24].

Since the extranonce ξ is also placed in the coinbase transaction, the ASIC is not able to plant it there only using the Merkle root z . Thus, ξ has to be fixed together with s when computing z . Therefore, in our attack, the ASIC only searches for a nonce ν , which translates to 2^{32} possible values. As we discuss in Sec. 4.3, for some ASICs, such reprogramming is feasible in practice. Before we do that, we prove that this attack breaks the security of the ASIC-ZKPoCK protocol of [24], i.e., that the values received by the ASIC do not reveal any information about the witness w . To be more formal, we need to assume that the underlying sigma protocol does not permit the reconstruction of the witness w from a single valid transcript (a, c, s) . Technically, this does not follow from the zero-knowledge property of the sigma protocol, since it could be the case that simply the relation $\mathcal{R}$ permits to reconstruct w from x . Therefore, we restrict our attention to relations $\mathcal{R}$ for which there exists a *hard instance generator*, defined below.

Definition 1 (Hard instance generator). Let $\mathcal{R} \subseteq \{0, 1\}^* \times \{0, 1\}^*$ be a binary relation. A probabilistic polynomial-time algorithm

$$\text{Gen}(1^\kappa)$$

is called a *hard instance generator* for $\mathcal{R}$ if it satisfies:

- For every security parameter κ , the algorithm outputs a pair

$$(x, w) \leftarrow \text{Gen}(1^\kappa)$$

such that $(x, w) \in \mathcal{R}$.

- For every probabilistic polynomial-time algorithm A , there exists a negligible function $\text{negl}(\kappa)$ such that

$$\Pr [(x, w') \in \mathcal{R} : (x, w) \leftarrow \text{Gen}(1^\kappa); w' \leftarrow A(1^\kappa, x)] \leq \text{negl}(\kappa).$$

For each round $i \in \{1, \dots, n\}$:

1. The prover P^* samples $(a, st) \leftarrow \mathcal{P}_1^\Sigma(x, w)$, sends a to V , and receives nonce r from V .
2. *Prover P :*
 - (a) Choose augmented context $\widehat{C}$ randomly.
 - (b) Let $c := F(r \parallel \widehat{C})$.
 - (c) Sample extranonce ξ^* randomly.
 - (d) Execute the following using secret computation using shared (or embedded in TEE) witness w :

i. Compute the sigma-response $s := \mathcal{P}_2^\Sigma(st, c, x, w)$. ii. Using s and ξ^*, compute the Merkle root z of the coinbase transaction with response s and extranonce ξ^* placed in it. iii. Output z.

 - (e) Using z compute B , where B is the block header template with response s and extranonce ξ^* placed in it. More precisely $B := C_s(\text{nonce} := \xi^*)$, where $C_s := \widehat{C}[\text{response} := s]$,
 - (f) Send B to the ASIC.
3. *ASIC:*
 - (a) Look for a nonce $\nu^* \in \{0, 1\}^{32}$ such that $(\xi^*, \nu^*) \in \mathcal{S}$ and $H(B \parallel \nu^*) < 2^{256-d}$.
 - (b) If you find such ν^* then return it; otherwise return “no hit”.
4. *Prover P :*
 - (a) If the ASIC returned a hit ν^* , send the *round solution* $(\widehat{C}, s, \xi^*, \nu^*)$ to the verifier and proceed to the next round.
 - (b) Else:
 - i. If the deadline τ is approaching, then finish the round without submitting a proof.
 - ii. Otherwise, go back to the step where you sample a fresh C_s (Step 2a).

Fig. 6: Adversarial mining procedure for ASIC-ZKPoCK

An example of a relation $\mathcal{R}$ that admits a hard instance generator is the discrete logarithm relation used, e.g., in the Schnorr protocol. Because hard instance generators are defined asymptotically, to instantiate them in practice, one needs to choose a security parameter κ . Therefore, we formulate the following lemma in an asymptotic way as well (assuming that the ASIC-ZKPoCK protocol is run for security parameter κ , denoting the hash output length).

Lemma 2. *Let 1^κ be the security parameter. Let Gen be a hard instance generator for a relation $\mathcal{R}$. Consider an ASIC-ZKPoCK protocol from Fig. 5 for $\mathcal{R}$ built using a sigma protocol $\mathcal{P} = (\mathcal{P}^\Sigma, \mathcal{V}^\Sigma)$ with challenge space $\mathcal{C}$ and hash function $F : \{0, 1\}^* \rightarrow \mathcal{C}$ modeled as a random oracle. Let $H : \{0, 1\}^* \rightarrow \{0, 1\}^\kappa$ be a hash function modeled as a random oracle. Let (x, w) be generated as $(x, w) \leftarrow \text{Gen}(1^\kappa)$.*

Consider an execution of the ASIC-ZKPoCK protocol in which the prover P^ follows the adversarial mining procedure from Fig. 6. Let $\{B_j\}_{j=1}^N$ be the sequence of all pairs sent by P^* to the ASIC during the execution (in Step 2f). Then for any poly-time eavesdropper $\mathcal{E}$ that takes as input a, x, r , and $\{B_j\}_{j=1}^N$, the probability that $\mathcal{E}$ outputs w is negligible in κ .*

Note that in the lemma statement, we model the hash function H as a random oracle, even though it works on input of arbitrary length, and hence it can be subject to precomputation attacks. This does not affect the relevance of the lemma, since it shows an *attack*, not a security proof.

Proof (of Lemma 1, sketch). Let Bad be the event that either

- P^* queries the random oracle containing H on a coinbase transaction containing the witness w , or
- $\mathcal{E}$ outputs w .

Clearly, at the before the execution of the protocol starts, **Bad** does not occur. Moreover, as long as **Bad** did not occur, the probability that it occurs in the next step of the algorithm is negligible in κ . This is because (a) the coinbase transaction is hashed using a random oracle, and consequently the B_j ’s give no information about w , and (b) that **Gen** is a hard instance generator. Therefore, the probability of **Bad** is negligible in κ , and hence the probability that $\mathcal{E}$ outputs w is negligible in κ . $\square$

	(a) hash-rate	(b) time in microseconds to replace the block header prefix	(c) number of BM1397 chips needed for the attack	(d) estimated cost of the attack in USD (USD 70 $\times$ the number of BM1397 chips)
Antminer S19 Pro Hydro	154 TH/s,	28	685	47,950
Antminer S9	13 TH/s	330	58	4,060

Table 2: Estimates for the practicality of our attack on ASIC-ZKPoCK executed on Antminer S19 Pro Hydro (mentioned in Appendix C.3 of [24]) and on Antminer S9 (used for the experiments in [24]). The table contains information about (a) the hashrate of the given miner, (b) the time needed to replace the block header prefix to match the hashrate, (c) the number of BM1397 chips needed to match the hashrate, and (d) the cost needed to mount our attack using these rigs. The estimates are based on the data from the top table, and the cost of a single BM1397 chip being 70 USD (the approximate price we paid). As explained in Sec. 4.3, these estimates rely on our experiments that demonstrate that the BM1397 chip achieved a hashrate of 225 GH/s when mining by changing the nonce only.

4.3 Implementation and discussion on the attack parameters

As described above, our attack uses the ASIC miner to search only through the nonce space, while the extranonce needs to be set externally to the miner. This raises a question whether such an attack is practical, i.e., whether the miner can achieve a high hashrate when searching only through the nonce space, and what bandwidth is needed to feed the miner with new block header prefixes. Table 2 summarizes the resources needed to mount the attack on the two mining rigs mentioned in [24], i.e., Antminer S19 Pro Hydro (with a hashrate of 154 TH/s) and Antminer S9 (13 TH/s). We performed experiments to estimate the feasibility of achieving these rates. Our experiments were performed on a Bitmain BM1397 ASIC chip. To obtain this chip, we purchased a Bitaxe Max miner (that contains it) for approximately USD 70. According to our benchmarks, the chip achieves a hashrate of around 225 GH/s when mining by changing only the nonce, and the block header prefix provided externally (this includes the time needed to transfer the prefix to the chip)¹⁴. The 225 GH/s hashrate is around 70% of the hashrate of the full Bitaxe Max miner (330 GH/s), which searches through both the nonce and extranonce spaces.

Table 2 includes information on the number and costs of BM1397 chips required to match the hashrates of the two mining rigs. The practicality of our attack depends on whether we use ZKPoCK as a countermeasure against the attacks by TEE or by MPC.

¹⁴ On the technical level, we do not actually transfer the block header prefix to the chip; instead, we send to it a midstate in the SHA-256 compression function. This does not affect the logic of our attack.

TEE attacks. When ZKPoCK is used as a countermeasure against TEE-based attacks (which is the main focus of [24]), the eavesdropper $\mathcal{E}$ is assumed to be able to run on a powerful CPU within a TEE enclave. The 225 GH/s hashrate achieved by our ASIC implementation is around 3,350 times faster than the hashrate of a state-of-the-art CPU estimated in [24, p. 11] to be around 2^{26} H/s. Since the computation of a new block header prefix requires a small number of hash computations, a CPU can compute new prefixes fast enough to keep up with the miner’s hashrate. This is true even when the CPU computation is done within the SGX. This refutes the assumption made in [24, p. 16], that there is no “practical resource” that prevents eavesdropping by $\mathcal{E}$ and works significantly faster than a CPU. In more concrete terms, since the attack can be parallelized, it translates to an attack with 685 BM1397 chips (for Antminer S19 Pro Hydro) or 58 BM1397 chips (for Antminer S9). The cost of our attack is below USD 50,000 and below USD 5,000, respectively (see Table 2). This is a very conservative estimate, as the real-life attacker could obtain the chips at a lower price when bought in bulk, or use more efficient mining chips.

MPC attacks. Launching an attack using MPC is more challenging. As shown in Table 2, even for Antminer S9, one would need to recompute around three block header prefixes every millisecond to keep up with the miner’s hashrate. Since each such computation requires three computations of the SHA-256 compression function, this would lead to high costs (also in terms of communication) even in the 2-party passive security setting. Still, given the current state of the art for MPC implementations of SHA-256, it does not seem downright impossible in the future. For Antminer S19 Pro Hydro, the requirements are even more challenging and probably out of reach for a long time.

4.4 Countermeasures

Let us now discuss possible countermeasures against the attacks described in Sec. 4.2. Depending on the use case, there are several countermeasures that enable implementing a protocol that is immune to the attacks described in Sec. 4.2. One option is to use the zero-knowledge proof of individual knowledge (zkPIK) protocol from [16]. However, the aim of the PoCK construction [24] is to protect against malicious use of TEEs (such as SGX) using *dedicated* hardware. In contrast, the construction of [16] is not compatible with the hardware available on the market.

Perhaps the most straightforward countermeasure is to modify the ZKPoCK protocol [24] to include the sigma-response s in a different part of the block header. Since Bitcoin mining relies on double SHA-256 hashing of the block header, we need to be careful to avoid the precomputation attacks similar to those described in Sec. 3. To be more specific, consider the structure of the block header depicted in Fig. 7. If, e.g., the response s is placed at the very beginning of the block header, then the cheating prover could precompute the midstate (i.e., the output of the first hash function in Fig. 7) and use it to mine by changing only the nonce, similarly to the attack described in Sec. 4.2.

One possibility would be to include s in the timestamp field of the block header. This, however, is problematic, as 32 bits are not sufficient to include the entire response s (which is 256 bits long). The most natural solution is to place s in the midstate, as it is exactly 256 bits long. As mentioned in Footnote 14, some mining ASICs (such as the BM1397) allow the midstate to be provided externally to the miner. A complete formalization and practicality analysis of this countermeasure will be provided in the extended version of this work.

References

- [1] J. Austgen, A. Fábrega, M. Kelkar, D. Vilardell, S. Allen, K. Babel, J. Yu, and A. Juels. “Liquefaction: Privately Liquefying Blockchain Assets”. In: *IEEE Symposium on Security and Privacy, SP 2025, San Francisco, CA, USA, May 12-15, 2025*. Ed. by M. Blanton, W. Enck, and C. Nita-Rotaru. IEEE, 2025, pp. 1493–1511. DOI: [10.1109/SP61157.2025.00156](https://doi.org/10.1109/SP61157.2025.00156).

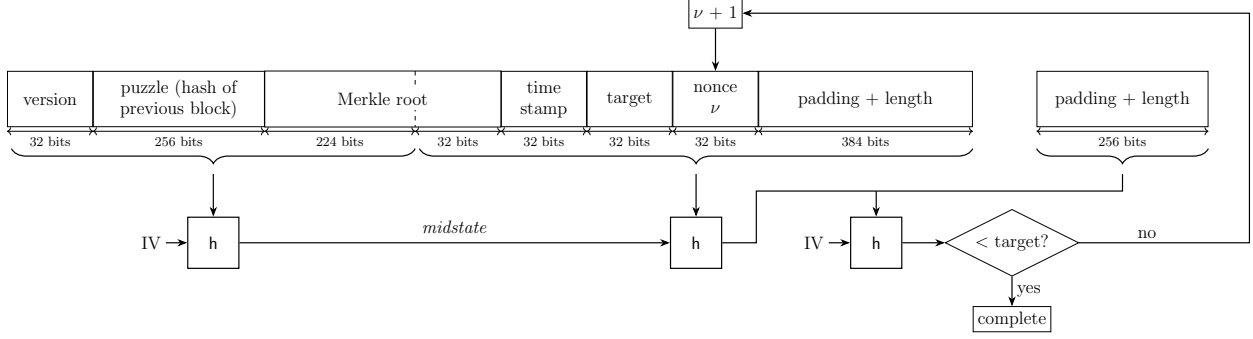


Fig. 7: Overview of the Bitcoin mining process. The block header is divided into two 512-bit parts, with the first part being first precomputed with h to produce the *midstate*, the second part of the block is then padded and hashed together with the *midstate*, and the result is again hashed using h to produce the final hash (notice that the Bitcoin hash uses *double* SHA-256 hashing).

- [2] S. Bano, A. Sonnino, M. Al-Bassam, S. Azouvi, P. McCorry, S. Meiklejohn, and G. Danezis. “SoK: Consensus in the Age of Blockchains”. In: *Proceedings of the 1st ACM Conference on Advances in Financial Technologies, AFT 2019, Zurich, Switzerland, October 21-23, 2019*. ACM, 2019, pp. 183–198. DOI: [10.1145/3318041.3355458](https://doi.org/10.1145/3318041.3355458).
- [3] M. Bellare, A. Boldyreva, and A. Palacio. “An Uninstantiable Random-Oracle-Model Scheme for a Hybrid-Encryption Problem”. In: *Advances in Cryptology - EUROCRYPT 2004, International Conference on the Theory and Applications of Cryptographic Techniques, Interlaken, Switzerland, May 2-6, 2004, Proceedings*. Ed. by C. Cachin and J. Camenisch. Vol. 3027. Lecture Notes in Computer Science. Springer, 2004, pp. 171–188. DOI: [10.1007/978-3-540-24676-3_11](https://doi.org/10.1007/978-3-540-24676-3_11).
- [4] M. Bellare and P. Rogaway. “Random oracles are practical: a paradigm for designing efficient protocols”. In: *Proceedings of the 1st ACM Conference on Computer and Communications Security, CCS ’93*. Fairfax, Virginia, USA: Association for Computing Machinery, 1993, pp. 62–73. ISBN: 0897916298. DOI: [10.1145/168588.168596](https://doi.org/10.1145/168588.168596). URL: <https://doi.org/10.1145/168588.168596>.
- [5] G. Bertoni, J. Daemen, M. Peeters, and G. Assche. “Sponge Functions”. In: *ECRYPT Hash Workshop 2007* (Jan. 2007).
- [6] J. Bormet, S. Dziembowski, S. Faust, T. Lizurej, and M. Mielniczuk. “Strong Secret Sharing with Snitching”. In: *Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2025, Proceedings, Part IV*. Ed. by Y. T. Kalai and S. F. Kamara. Vol. 16003. Lecture Notes in Computer Science. Springer, 2025, pp. 295–327. DOI: [10.1007/978-3-032-01884-7_10](https://doi.org/10.1007/978-3-032-01884-7_10).
- [7] K. D. Bowers, A. Juels, and A. Oprea. “Proofs of retrievability: theory and implementation”. In: *Proceedings of the first ACM Cloud Computing Security Workshop, CCSW 2009, Chicago, IL, USA, November 13, 2009*. Ed. by R. Sion and D. Song. ACM, 2009, pp. 43–54. DOI: [10.1145/1655008.1655015](https://doi.org/10.1145/1655008.1655015).
- [8] R. Canetti, O. Goldreich, and S. Halevi. “The random oracle methodology, revisited”. In: *J. ACM* 51.4 (2004), pp. 557–594. DOI: [10.1145/1008731.1008734](https://doi.org/10.1145/1008731.1008734).
- [9] A. Chepurnoy and A. Saxena. “Bypassing Non-Outsourceable Proof-of-Work Schemes Using Collateralized Smart Contracts”. In: *Financial Cryptography and Data Security - FC 2020 International Workshops, AsiaUSEC, CoDeFi, VOTING, and WTSC, Kota Kinabalu, Malaysia, February 14, 2020, Revised Selected Papers*. Ed. by M. Bernhard, A. Bracciali, L. J. Camp, S. Matsuo, A. Maurushat, P. B. Rønne, and M. Sala. Vol. 12063. Lecture Notes in Computer Science. Springer, 2020, pp. 423–435. DOI: [10.1007/978-3-030-54455-3_30](https://doi.org/10.1007/978-3-030-54455-3_30).

- [10] J. H. Chiang, B. David, T. K. Frederiksen, A. Mondal, and E. Yeniaras. “Detecting Rogue Decryption in (Threshold) Encryption via Self-Incriminating Proofs”. In: *IACR Cryptol. ePrint Arch.* (2024), p. 794. URL: <https://eprint.iacr.org/2024/794>.
- [11] R. Cramer, I. B. Damgård, and J. B. Nielsen. *Secure Multiparty Computation and Secret Sharing*. Cambridge University Press, 2015.
- [12] P. Daian, I. Eyal, A. Juels, and E. G. Sirer. “(Short Paper) PieceWork: Generalized Outsourcing Control for Proofs of Work”. In: *Financial Cryptography and Data Security - FC 2017 International Workshops, WAHC, BITCOIN, VOTING, WTSC, and TA, Sliema, Malta, April 7, 2017, Revised Selected Papers*. Ed. by M. Brenner, K. Rohloff, J. Bonneau, A. Miller, P. Y. A. Ryan, V. Teague, A. Bracciali, M. Sala, F. Pintore, and M. Jakobsson. Vol. 10323. Lecture Notes in Computer Science. Springer, 2017, pp. 182–190. DOI: [10.1007/978-3-319-70278-0_11](https://doi.org/10.1007/978-3-319-70278-0_11).
- [13] I. B. Damgård. “A Design Principle for Hash Functions”. In: *Advances in Cryptology — CRYPTO’89 Proceedings*. Ed. by G. Brassard. New York, NY: Springer New York, 1990, pp. 416–427. ISBN: 978-0-387-34805-6.
- [14] C. Dwork and M. Naor. “Pricing via Processing or Combatting Junk Mail”. In: *Advances in Cryptology - CRYPTO ’92, 12th Annual International Cryptology Conference, Santa Barbara, California, USA, August 16-20, 1992, Proceedings*. Ed. by E. F. Brickell. Vol. 740. Lecture Notes in Computer Science. Springer, 1992, pp. 139–147. DOI: [10.1007/3-540-48071-4_10](https://doi.org/10.1007/3-540-48071-4_10).
- [15] S. Dziembowski, S. Faust, V. Kolmogorov, and K. Pietrzak. “Proofs of Space”. In: *Advances in Cryptology - CRYPTO 2015 - 35th Annual Cryptology Conference, Santa Barbara, CA, USA, August 16-20, 2015, Proceedings, Part II*. Ed. by R. Gennaro and M. Robshaw. Vol. 9216. Lecture Notes in Computer Science. Springer, 2015, pp. 585–605. DOI: [10.1007/978-3-662-48000-7_29](https://doi.org/10.1007/978-3-662-48000-7_29).
- [16] S. Dziembowski, S. Faust, and T. Lizeur. “Individual Cryptography”. In: *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part II*. Ed. by H. Handschuh and A. Lysyanskaya. Vol. 14082. Lecture Notes in Computer Science. Springer, 2023, pp. 547–579. DOI: [10.1007/978-3-031-38545-2_18](https://doi.org/10.1007/978-3-031-38545-2_18).
- [17] S. Dziembowski, S. Faust, T. Lizeur, and M. Mielniczuk. “Secret Sharing with Snitching”. In: *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. CCS ’24. Salt Lake City, UT, USA: Association for Computing Machinery, 2024, pp. 840–853. ISBN: 9798400706363. DOI: [10.1145/3658644.3690296](https://doi.org/10.1145/3658644.3690296). URL: <https://doi.org/10.1145/3658644.3690296>.
- [18] A. Fábrega, A. Zhao, J. Yu, J. Austgen, S. Allen, K. Babel, M. Kelkar, and A. Juels. “Voting-Bloc Entropy: A New Metric for DAO Decentralization”. In: *34th USENIX Security Symposium, USENIX Security 2025, Seattle, WA, USA, August 13-15, 2025*. Ed. by L. Bauer and G. Pellegrino. USENIX Association, 2025, pp. 1299–1318. URL: <https://www.usenix.org/conference/usenixsecurity25/presentation/fabrega-entropy>.
- [19] B. Fisch. “Tight Proofs of Space and Replication”. In: *Advances in Cryptology - EUROCRYPT 2019 - 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Darmstadt, Germany, May 19-23, 2019, Proceedings, Part II*. Ed. by Y. Ishai and V. Rijmen. Vol. 11477. Lecture Notes in Computer Science. Springer, 2019, pp. 324–348. DOI: [10.1007/978-3-030-17656-3_12](https://doi.org/10.1007/978-3-030-17656-3_12).
- [20] S. Goldwasser, S. Micali, and C. Rackoff. “The Knowledge Complexity of Interactive Proof Systems”. In: *SIAM J. Comput.* 18.1 (1989), pp. 186–208. DOI: [10.1137/0218012](https://doi.org/10.1137/0218012).
- [21] R. Han, H. Lin, and J. Yu. “VRF-Based Mining Simple Non-outsourced Cryptocurrency Mining”. In: *Data Privacy Management, Cryptocurrencies and Blockchain Technology - ESORICS 2020 International Workshops, DPM 2020 and CBT 2020, Guildford, UK, September 17-18, 2020, Revised Selected Papers*. Vol. 12484. Lecture Notes in Computer Science. Springer, 2020, pp. 287–304. DOI: [10.1007/978-3-030-66172-4_19](https://doi.org/10.1007/978-3-030-66172-4_19).
- [22] C. Hazay and Y. Lindell. “Sigma Protocols and Efficient Zero-Knowledge”. In: *Efficient Secure Two-Party Protocols: Techniques and Constructions*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010,

- pp. 147–175. ISBN: 978-3-642-14303-8. DOI: [10.1007/978-3-642-14303-8_6](https://doi.org/10.1007/978-3-642-14303-8_6). URL: https://doi.org/10.1007/978-3-642-14303-8_6.
- [23] J. Katz and Y. Lindell. *Introduction to Modern Cryptography, Second Edition*. CRC Press, 2014. ISBN: 9781466570269. URL: <https://www.crcpress.com/Introduction-to-Modern-Cryptography-Second-Edition/Katz-Lindell/p/book/9781466570269>.
 - [24] M. Kelkar, K. Babel, P. Daian, J. Austgen, V. Buterin, and A. Juels. “Complete Knowledge: Preventing Encumbrance of Cryptographic Secrets”. In: *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, Salt Lake City, UT, USA, October 14-18, 2024*. Ed. by B. Luo, X. Liao, J. Xu, E. Kirda, and D. Lie. ACM, 2024, pp. 2415–2429. DOI: [10.1145/3658644.3690273](https://doi.org/10.1145/3658644.3690273).
 - [25] D. Khovratovich, R. D. Rothblum, and L. Soukhanov. “How to Prove False Statements: Practical Attacks on Fiat-Shamir”. In: *IACR Cryptol. ePrint Arch.* (2025), p. 118. URL: <https://eprint.iacr.org/2025/118>.
 - [26] G. Leurent and P. Q. Nguyen. “How Risky Is the Random-Oracle Model?” In: *Advances in Cryptology - CRYPTO 2009, 29th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 16-20, 2009. Proceedings*. Ed. by S. Halevi. Vol. 5677. Lecture Notes in Computer Science. Springer, 2009, pp. 445–464. DOI: [10.1007/978-3-642-03356-8_26](https://doi.org/10.1007/978-3-642-03356-8_26).
 - [27] U. M. Maurer, R. Renner, and C. Holenstein. “Indifferentiability, Impossibility Results on Reductions, and Applications to the Random Oracle Methodology”. In: *Theory of Cryptography, First Theory of Cryptography Conference, TCC 2004, Cambridge, MA, USA, February 19-21, 2004, Proceedings*. Ed. by M. Naor. Vol. 2951. Lecture Notes in Computer Science. Springer, 2004, pp. 21–39. DOI: [10.1007/978-3-540-24638-1_2](https://doi.org/10.1007/978-3-540-24638-1_2).
 - [28] R. C. Merkle. “A Certified Digital Signature”. In: *Advances in Cryptology — CRYPTO’89 Proceedings*. Ed. by G. Brassard. New York, NY: Springer New York, 1990, pp. 218–238. ISBN: 978-0-387-34805-6.
 - [29] R. C. Merkle. “A Digital Signature Based on a Conventional Encryption Function”. In: *Advances in Cryptology - CRYPTO ’87, A Conference on the Theory and Applications of Cryptographic Techniques, Santa Barbara, California, USA, August 16-20, 1987, Proceedings*. Ed. by C. Pomerance. Vol. 293. Lecture Notes in Computer Science. Springer, 1987, pp. 369–378. DOI: [10.1007/3-540-48184-2_32](https://doi.org/10.1007/3-540-48184-2_32).
 - [30] A. Miller, A. E. Kosba, J. Katz, and E. Shi. “Nonoutsourcable Scratch-Off Puzzles to Discourage Bitcoin Mining Coalitions”. In: *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security, Denver, CO, USA, October 12-16, 2015*. Ed. by I. Ray, N. Li, and C. Kruegel. ACM, 2015, pp. 680–691. DOI: [10.1145/2810103.2813621](https://doi.org/10.1145/2810103.2813621).
 - [31] A. Miller, A. E. Kosba, J. Katz, and E. Shi. *Nonoutsourcable Scratch-Off Puzzles to Discourage Bitcoin Mining Coalitions*. Full version of [31] available at https://soc1024.ece.illinois.edu/nonoutsourcable_full.pdf. 2015.
 - [32] A. Narayanan, J. Bonneau, E. Felten, A. Miller, and S. Goldfeder. *Bitcoin and Cryptocurrency Technologies: A Comprehensive Introduction*. USA: Princeton University Press, 2016. ISBN: 0691171696.
 - [33] M. Rosenfeld. “Analysis of Bitcoin Pooled Mining Reward Systems”. In: *CoRR* abs/1112.4980 (2011). DOI: [10.48550/arxiv.1112.4980](https://doi.org/10.48550/arxiv.1112.4980). arXiv: 1112.4980. URL: <http://arxiv.org/abs/1112.4980>.
 - [34] S. Ruj, S. Kanhere, and M. Conti. *Blockchains: A Handbook on Fundamentals, Platforms and Applications*. Advances in Information Security. Springer International Publishing, 2024. ISBN: 9783031321467.
 - [35] C. Schnorr. “Efficient Identification and Signatures for Smart Cards”. In: *Advances in Cryptology - CRYPTO ’89, 9th Annual International Cryptology Conference, Santa Barbara, California, USA, August 20-24, 1989, Proceedings*. Ed. by G. Brassard. Vol. 435. Lecture Notes in Computer Science. Springer, 1989, pp. 239–252. DOI: [10.1007/0-387-34805-0_22](https://doi.org/10.1007/0-387-34805-0_22).
 - [36] A. Shamir. “How to Share a Secret”. In: *Commun. ACM* 22.11 (1979), pp. 612–613. DOI: [10.1145/359168.359176](https://doi.org/10.1145/359168.359176).
 - [37] W. Wang, D. T. Hoang, P. Hu, Z. Xiong, D. Niyato, P. Wang, Y. Wen, and D. I. Kim. “A Survey on Consensus Mechanisms and Mining Strategy Management in Blockchain Networks”. In: *IEEE Access* 7 (2019), pp. 22328–22370. DOI: [10.1109/ACCESS.2019.2896108](https://doi.org/10.1109/ACCESS.2019.2896108).

- [38] Wikipedia contributors. *Secure Hash Algorithms* — *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/w/index.php?title=Secure_Hash_Algorithms&oldid=1310574193. [Online; accessed 9-October-2025]. 2025.
- [39] Wikipedia contributors. *SHA-2* — *Wikipedia, The Free Encyclopedia*. <https://en.wikipedia.org/w/index.php?title=SHA-2&oldid=1303317086>. [Online; accessed 9-August-2025]. 2025.
- [40] Wikipedia contributors. *Trusted execution environment* — *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/w/index.php?title=Trusted_execution_environment&oldid=1292922440. [Online; accessed 4-June-2025]. 2025.
- [41] J. Xu, C. Wang, and X. Jia. “A Survey of Blockchain Consensus Protocols”. In: *ACM Comput. Surv.* 55.13s (2023), 278:1–278:35. DOI: [10.1145/3579845](https://doi.org/10.1145/3579845).
- [42] G. Zeng, Z. Huang, Y. Wang, X. Mu, and C. Zhang. “Strongly nonoutsourcable scratch-off puzzles in blockchain”. In: *Soft Comput.* 27.17 (2023), pp. 11941–11960. DOI: [10.1007/S00500-023-08753-1](https://doi.org/10.1007/S00500-023-08753-1).
- [43] G. Zeng, S. Yiu, J. Zhang, H. Kuzuno, and M. H. Au. “A Nonoutsourcable Puzzle Under GHOST Rule”. In: *15th Annual Conference on Privacy, Security and Trust, PST 2017, Calgary, AB, Canada, August 28-30, 2017*. IEEE Computer Society, 2017, pp. 35–43. DOI: [10.1109/PST.2017.00015](https://doi.org/10.1109/PST.2017.00015).